

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design
On

Honeypot Network for EdTech Threat Intelligence

Student Name: Naveen S Sreedhar

Roll Number: 23BCCE125

Guided by: Vijayalaxmi

Academic Year: 2023 – 2026

Table of Contents

1. Introduction	3
1.1. Scope of the Document	3
1.2. Intended Audience	3
1.3. System Overview	3
1.4. Assumptions	4
2. System Design	4
2.1. Application Design	4
2.2. Process Flow	5
2.3. Information Flow	5
2.4. Components Design	6
2.5. Key Design Considerations	6
2.6. API Catalogue	7
3. Data Design	7
3.1. Data Model	7
3.2. Data Access Mechanism	8
3.3. Data Retention Policies	8
3.4. Data Migration	8
4. Interfaces	9
5. State and Session Management	9
6. Caching	10
7. Non-Functional Requirements	10
7.1. Security Aspects	10
7.2. Performance Aspects	11
7.3. Limitations	11
7.4. Future Enhancements	11
7.5. Testing Summary	12
8. References	12

1. Introduction

This document presents the High-Level Design (HLD) of the Honeypot Network for EdTech Threat Intelligence system. The system is designed to listen on a dedicated port, log every connection attempt in real time, send a fake 'Access Denied' banner to the attacker, and present all captured threat data through a web-based monitoring dashboard. It is inspired by the working principle of enterprise honeypot tools such as Cowrie and T-Pot used by real security teams.

The project simulates a Security Operations Center (SOC) workflow — from the moment an attacker attempts a connection to the moment the incident is recorded and displayed on the dashboard — making it suitable for academic study and real-world awareness in EdTech cybersecurity.

1.1. Scope of the Document

This HLD document covers the following aspects of the system:

- Overall system architecture and component interactions
- Application design including process flow and information flow
- Data design including log storage format, access mechanism and retention policies
- Non-functional requirements such as security and performance
- Interface design for the Flask-based dashboard and fake EdTech portal

This document does not cover low-level implementation details such as line-by-line code logic, which are addressed in the Low-Level Design (LLD) document.

1.2. Intended Audience

- Academic evaluators and faculty reviewing the project submission
- Developers or students who may extend or build upon this system
- Cybersecurity enthusiasts seeking to understand honeypot-based threat intelligence
- Anyone interested in the application of Python sockets in network security

1.3. System Overview

The Honeypot Network for EdTech Threat Intelligence is a real-time cybersecurity monitoring system built using Python, the socket library, threading, and Flask. It listens on port 8888, captures every connection attempt with IP and timestamp, logs the data to CSV and JSON files, responds with a fake EduPortal server banner, and presents all threat data through an interactive web-based dashboard.

The system mimics the workflow of a real SOC honeypot environment:

- Listen: Bind to port 8888 and accept all incoming TCP connections
- Detect: Record attacker IP address, port, payload data, and timestamp
- Trap: Capture login credentials via the fake EduNova portal login form
- Flag: Auto-flag any visitor to hidden trap URLs (/admin, /.env, /phpmyadmin etc.)
- Respond: Send a fake HTTP 403 Access Denied banner with EduPortal-v2.1 server header
- Visualize: Display real-time attack statistics, charts and live log table on dashboard

Component	Technology	Purpose
Port Listener	Python Socket Library	Binds to port 8888, accepts all TCP connections

Component	Technology	Purpose
Concurrency	Python Threading	Handles multiple connections simultaneously
Logging	CSV + JSON File I/O	Stores IP, port, payload, timestamp records
Login Trap	Flask POST /login	Captures attacker email and password attempts
Fake Response	Python Bytes Encoding	Sends fake Access Denied banner to attacker
Dashboard	Flask + HTML + CSS	Real-time web monitoring interface
Visualization	Chart.js	Bar charts for hourly attacks and top attacker IPs

1.4. Assumptions

- Python3.x environment is configured on the host system
- Required library (Flask) is installed via pip
- Application runs on localhost or internal WiFi network during demo
- User has permission to bind to port 8888
- Log directory (logs/) is writable for persistent record keeping
- Both honeypot.py and dashboard.py run in separate terminal instances

2. System Design

2.1. Application Design

This system is built as two cooperating Python applications running side by side. The core honeypot (honeypot.py) is a multi-threaded TCP socket server that binds to port 8888 and listens for connections indefinitely. The dashboard (dashboard.py) is a separate Flask web server that serves the fake EduNova portal, captures login attempts, and provides a threat intelligence dashboard. Both applications share state only through the CSV and JSON log files, keeping them fully decoupled.

2.2. Process Flow

The process flow of the system follows a sequential pipeline:

- Step 1 — Initialization: Port configuration loaded from config.py, log files created on startup
- Step 2 — Binding: Socket bound to 0.0.0.0:8888 using SO_REUSEADDR for instant restart
- Step 3 — Threading: One daemon thread spawned per connection for parallel handling
- Step 4 — Accept: Main loop calls accept() continuously on the server socket
- Step 5 — Logging: On connection, IP, port, payload data and timestamp are captured
- Step 6 — Detection: Sliding window algorithm checks if IP has 3+ connections in 60 seconds
- Step 7 — Persistence: Entry appended to logs/connections.csv and suspicious.json if flagged
- Step 8 — Response: Fake HTTP 403 Access Denied banner sent to attacker, connection closed
- Step 9 — Trap URLs: Any visit to /admin, /.env, /phpmyadmin etc. auto-flags the IP immediately
- Step 10 — Visualization: Flask dashboard reads log files and renders threat intelligence UI

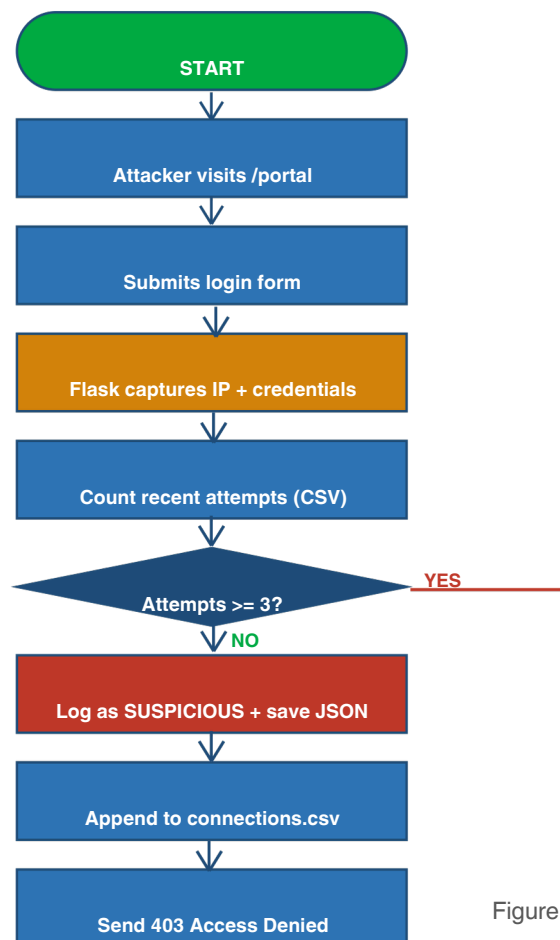


Figure 1.1 — Process Flow Diagram

2.3. Information Flow

Dataflow through the system in the following direction:

- Attacker → TCP connection to honeypot port 8888 OR visits fake EduNova portal
- Socket listener → accepts connection, hands off to dedicated handler thread
- Connection handler → extracts IP address, port, payload data and `datetime.now()`
- Detector → checks previous attempts within 60-second sliding window from CSV
- Logger → appends entry {timestamp, ip, port, data, is_suspicious} to `connections.csv`
- Suspicious logger → if flagged, writes enriched entry to `suspicious.json`
- Fake banner → HTTP 403 with EduPortal-v2.1 Server header sent to attacker
- Flask server → reads CSV and JSON files on every dashboard page request
- Browser dashboard → displays live charts, stat cards, and connection table

2.4. Components Design

The system is composed of the following major components:

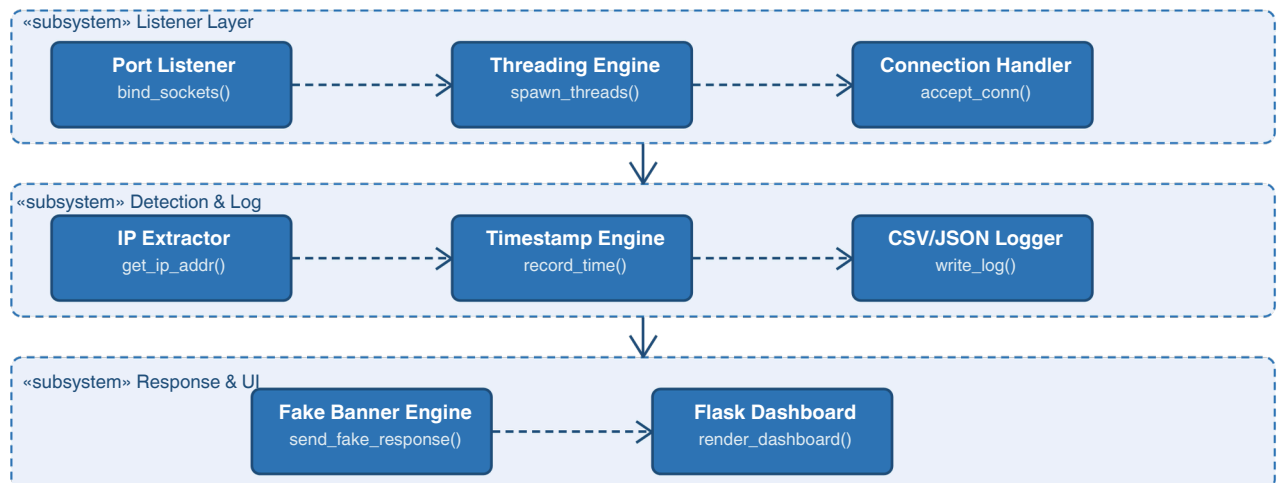


Figure 1.2 — UML Component Interaction Diagram

Component	Description
honeypot.py	Core TCP server — binds port 8888, spawns threads per connection, coordinates logger and detector
detector.py	ConnectionDetector class — sliding window algorithm, thread-safe via Lock, flags IPs after 3 attempts
logger.py	Writes CSV rows and JSON objects, creates log directories, generates summary report on shutdown
config.py	Centralized settings — HOST, PORT, thresholds, file paths. Single source of truth for all parameters
dashboard.py	Flask server — serves /portal fake website, /login trap, / dashboard, 15 hacker trap URLs, /api/stats

Component	Description
edunova_portal.html	Realistic fake EdTech website — 6 course cards, features, login form that POSTs to /login honeypot trap
Hacker Trap URLs	15 hidden URLs (/admin, /.env, /wp-admin, /phpmyadmin, /.git) — any visitor auto-flagged immediately

2.5. Key Design Considerations

- Allconnection handlers run as daemon threads — shut down cleanly when main process exits
- SO_REUSEADDR set on every socket to allow instant restart without 'Address already in use' errors
- Fake banner includes realistic Server: EduPortal-v2.1 header to prevent service fingerprinting
- Dashboard reads CSV directly for suspicious detection — accurate across server restarts
- Hacker trap URLs flag visitors immediately without waiting for 3-attempt threshold
- Log files stored as CSV and JSON — consumable directly by any SIEM platform
- Honeypot and dashboard are decoupled — only connected via shared log files
- host=0.0.0.0 allows access from any device on the same WiFi network for demo purposes

2.6. API Catalogue

The system exposes the following REST endpoints through the Flask dashboard:

Endpoint	Method	Response	Description
/portal	GET	HTML	Serves the fake EduNova EdTech login portal to lure attackers
/login	POST	403 HTML	Captures credentials — logs IP, email, password — returns Access Denied
/	GET	HTML	Renders the threat intelligence dashboard with live stats and charts
/admin	GET	401 HTML	Hacker trap — auto-flags visitor — returns fake admin login page
/.env	GET	200 Text	Hacker trap — returns fake WordPress admin panel page
/wp-admin	GET	401 HTML	Hacker trap — returns fake database console message
/phpmyadmin	GET	403 HTML	Hacker trap — returns fake git repository error response
/.git	GET	403 Text	Returns total connections, unique IPs, and suspicious count
/api/stats	GET	JSON	Returns list of all 15 active trap URLs
/api/traps	GET	JSON	

3. Data Design

3.1. Data Model

The system uses two flat file formats for data persistence. No relational database is required for this proof-of-concept implementation. The following internal function interfaces are defined:

Function	Input	Output	Description
setup_directories()	None	None	Creates logs/ and reports/ folders if not present
log_connection(ip,port,data,flag)	String,Int,Str,Bool	None	Appends one row to connections.csv
log_suspicious_ip(ip,count,times)	String,Int,List	None	Writes enriched IP entry to suspicious.json
generate_summary()	None	None	Writes summary report to reports/summary.txt
record_connection(ip)	String	Bool,Int	Checks sliding window, returns is_suspicious flag
count_recent_attempts(ip,secs)	String,Int	Int	Counts CSV rows for IP within time window

Primary data schema — connections.csv:

Field	Data Type	Example	Description
timestamp	String (YYYY-MM-DD HH:MM:SS)	2026-04-22 11:46:25	Date and time of connection
ip_address	String	192.168.6.210	Source IP address of attacker
port	Integer	80	Source port number used
data_received	String (max 200 chars)	LOGIN email=admin@..	Payload or description captured
is_suspicious	Boolean	True / False	Flagged by detector algorithm

3.2. Data Access Mechanism

- LogFile: logs/connections.csv — plain CSV array appended on each connection
- Append Strategy: new row appended using csv.writer in 'a' (append) mode — O(1) per write
- Dashboard Reads: Flask reads CSV using csv.DictReader, parses rows into Python dicts
- Suspicious File: logs/suspicious.json — full file loaded, updated, and rewritten on each flag
- Thread Safety: threading.Lock() in detector.py prevents concurrent write corruption

3.3. Data Retention Policies

- LogFile:connections.csv retained indefinitely on disk — never auto-deleted during operation

- In-Memory State: detector resets on restart — CSV provides full persistent history
- Dashboard Display: shows last 50 connection records in live table for readability
- Full Data: complete log files remain available for export and offline analysis in Excel

3.4. Data Migration

The system does not involve database migration — the log store is a flat CSV file. If the system is moved to a new host, copying the logs/ folder is sufficient to preserve the full attack history. If required, the same data can be imported into PostgreSQL, MongoDB, or an ELK stack for production deployments without modifying the honeypot core.

4. Interfaces

The system provides two web-based interfaces through the Flask dashboard, both accessible on the local network via `http://<host-ip>:5000`.

UI Element	Type	Description
IBM + NCS Header Bar	HTML header	Branding bar present on every page of the dashboard
Status Badge	HTML div	Shows MONITORING ACTIVE with pulsing green dot
Stat Cards (x4)	HTML cards	Total Hits, Unique IPs, Suspicious IPs, Normal Connections
Connections by Hour	Chart.js bar	Vertical bar chart showing attack frequency per hour
Top Attacker IPs	Chart.js bar	Horizontal bar chart showing most active attacker IPs
Live Connection Log	HTML table	Last 50 connections with timestamp, IP, port, status badge
Flagged IPs Panel	HTML list	Detailed view of all suspicious IPs with attempt counts
EduNova Portal	Full HTML page	Fake EdTech website with login form — the primary honeypot bait
Access Denied Page	HTML page	Displayed to attacker after login attempt with fake rejection

5. State and Session Management

The honeypot system does not maintain user sessions as it has no legitimate user login. State is managed through the following components:

- Listener State: daemon thread holds its own socket object and accept loop
- Connection State: every incoming connection handled independently — no session carry-over
- Detector State: in-memory IP timestamp dictionary — resets on restart, CSV provides persistence
- Log State: persistent across restarts — lives on disk in `connections.csv` and `suspicious.json`
- Dashboard State: stateless HTTP — every request re-reads the log files from scratch

6. Caching

The system does not use an in-memory cache for log data. Every dashboard page request reads the CSV and JSON log files fresh from disk. This keeps the implementation simple and guarantees the dashboard always reflects the latest recorded events. For production deployments at scale, a Redis layer could be added between the log files and Flask to avoid repeated disk reads — this is intentionally out of scope for the academic version.

7. Non-Functional Requirements

7.1. Security Aspects

- Fake banner hides all real server information — Server: EduPortal-v2.1 header misleads attackers
- Log files stored locally — not exposed to any external interface
- Dashboard accessible on local network only — no public internet exposure in academic version
- All trap URLs return realistic fake responses to prevent service fingerprinting
- No real student data exists on the honeypot server — zero data breach risk
- Input sanitization — all received data truncated to 200 characters before CSV write

7.2. Performance Aspects

- Multi-threaded handlers allow concurrent connections — socket backlog of 50
- Daemon threads ensure clean shutdown without zombie processes on Ctrl+C
- CSV append-write pattern — O(1) per write, suitable for lab to medium attack volumes
- 2-second recv() timeout prevents thread starvation on silent browser connections
- SO_REUSEADDR enables instant socket rebinding after restart
- host=0.0.0.0 allows WiFi network access for multi-device project demos

7.3. Limitations

- Local demo deployment only — no public IP in academic version
- CSV file storage not suitable for high-volume production traffic
- No geolocation of attacker IPs in this version
- No real-time alerting — purely passive logging and visualization
- In-memory detector state resets on server restart

7.4. Future Enhancements

- Deploy on cloud server (AWS/DigitalOcean) with public IP to attract real attackers
- Add IP geolocation to show a world map of attack origins
- Send email or Telegram alerts on repeated attack attempts
- Migrate storage from CSV/JSON to PostgreSQL or Elasticsearch
- Add fake SSH (port 22), fake MySQL (port 3306) for richer deception layers
- Integrate with Wazuh or Splunk for real SOC workflows
- Add machine learning classifier for attack type identification from payload data

7.5. Testing Summary

- Port 8888 successfully bound and listening — confirmed via test_honeypot.py
- All 4 test suites passed: single connection, repeated connections, 8 concurrent threads, varied payloads
- Login trap successfully captured email and password from browser form submission
- Fake Access Denied banner received correctly by test clients and browser
- Hacker trap URLs (/admin, /.env, /wp-admin) triggered immediate suspicious flag
- Dashboard updated correctly with accurate stats, charts, and flagged IP details
- Log files persisted correctly across honeypot restarts — CSV data retained
- System accessible from phone browser via WiFi — confirmed at http://192.168.6.210:5000

8. References

- Python Socket Library Documentation — <https://docs.python.org/3/library/socket.html>
- Python Threading Documentation — <https://docs.python.org/3/library/threading.html>
- Flask Official Documentation — <https://flask.palletsprojects.com>
- Chart.js Documentation — <https://www.chartjs.org/docs>
- OWASP Honeypot Project — <https://owasp.org/www-project-honeypot>
- Cowrie Honeypot Reference — <https://github.com/cowrie/cowrie>
- T-Pot Multi-Honeypot Platform — <https://github.com/telekom-security/tpotce>
- MITRE ATT&CK; Framework — <https://attack.mitre.org>
- Spitzner, L. (2002). Honeypots: Tracking Hackers. Addison-Wesley Professional.
- CISA (2023). Cybersecurity Risks in the Education Sector — <https://www.cisa.gov/>